

Naive Bayes Implementation Project

September 7, 2026

1 Naive Bayes - Implementation with Scikit-Learn:

1.1 Created by Abdullah Shaheer

```
[4]: # =====  
# 1) Import Required libraries  
# =====  
  
import numpy as np  
import pandas as pd  
  
from sklearn.datasets import load_breast_cancer  
from sklearn.model_selection import train_test_split  
from sklearn.naive_bayes import GaussianNB  
from sklearn.metrics import accuracy_score, confusion_matrix,  
    ↪classification_report  
  
# =====  
# 2) Load Dataset  
# =====  
  
data = load_breast_cancer()  
  
# Feature and Target  
  
x = data.data  
y = data.target  
  
# Convert to DataFrame for data feeling and better understanding  
  
df = pd.DataFrame(x, columns=data.feature_names)  
df['target'] = y  
  
print("Dataset Shape", df.shape)  
print("\nFirst 5 rows of dataset:\n")  
print(df.head())
```

Dataset Shape (569, 31)

First 5 rows of dataset:

	mean radius	mean texture	mean perimeter	mean area	mean smoothness \
0	17.99	10.38	122.80	1001.0	0.11840
1	20.57	17.77	132.90	1326.0	0.08474
2	19.69	21.25	130.00	1203.0	0.10960
3	11.42	20.38	77.58	386.1	0.14250
4	20.29	14.34	135.10	1297.0	0.10030

	mean compactness	mean concavity	mean concave points	mean symmetry \
0	0.27760	0.3001	0.14710	0.2419
1	0.07864	0.0869	0.07017	0.1812
2	0.15990	0.1974	0.12790	0.2069
3	0.28390	0.2414	0.10520	0.2597
4	0.13280	0.1980	0.10430	0.1809

	mean fractal dimension ...	worst texture	worst perimeter	worst area \
0	0.07871 ...	17.33	184.60	2019.0
1	0.05667 ...	23.41	158.80	1956.0
2	0.05999 ...	25.53	152.50	1709.0
3	0.09744 ...	26.50	98.87	567.7
4	0.05883 ...	16.67	152.20	1575.0

	worst smoothness	worst compactness	worst concavity	worst concave points \
0	0.1622	0.6656	0.7119	0.2654
1	0.1238	0.1866	0.2416	0.1860
2	0.1444	0.4245	0.4504	0.2430
3	0.2098	0.8663	0.6869	0.2575
4	0.1374	0.2050	0.4000	0.1625

	worst symmetry	worst fractal dimension	target
0	0.4601	0.11890	0
1	0.2750	0.08902	0
2	0.3613	0.08758	0
3	0.6638	0.17300	0
4	0.2364	0.07678	0

[5 rows x 31 columns]

```
[5]: # =====
# 3) Data information:
# =====

print("\nTarget Classes:")
print(data.target_names)
```

```

print("\nClass Distribution:")
print(df['target'].value_counts())

# =====
# 4) Split Dataset into Training and testing sets
# =====

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y
)

print("\nTraining Samples:", x_train.shape[0])

print("\nTesting Samples:", x_test.shape[0])

```

Target Classes:
['malignant' 'benign']

Class Distribution:
target
1 357
0 212
Name: count, dtype: int64

Training Samples: 426

Testing Samples: 143

```

[6]: # =====
# 5) Create and Train Naive Bayes Model
# =====
nb_model = GaussianNB()

# Train the model

nb_model.fit(x_train, y_train)

print("\nNaive Bayes model training completed.")

```

Naive Bayes model training completed.

```

[7]: # =====
# 6) make predictions
# =====
y_pred = nb_model.predict(x_test)

```

```
print("\nFirst 10 prediction:")
print(y_pred[:10])
```

First 10 prediction:
[1 0 1 1 1 0 0 0 0 1]

```
[8]: # =====
# 7) Model Evaluation.
# =====

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nModel Accuracy:", round(accuracy * 100, 2), "%")

# Confusion Matrix:

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
# Detailed Classification metric

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Model Accuracy: 93.71 %

Confusion Matrix:

```
[[46  7]
 [ 2 88]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.96	0.87	0.91	53
1	0.93	0.98	0.95	90
accuracy			0.94	143
macro avg	0.94	0.92	0.93	143
weighted avg	0.94	0.94	0.94	143

```
[9]: # =====
# 8) Interpretation of Confusion Matrix:
# =====

print("\nConfusion Matrix Interpretation:")
```

```

print("Ture Negative (TN):", cm[0][0])
print("False Positive (FP):", cm[0][1])
print("Flase Negitive (FN):", cm[1][0])
print("True Positive (TP):", cm[1][1])

```

Confusion Matrix Interpretation:

Ture Negative (TN): 46

False Positive (FP): 7

Flase Negitive (FN): 2

True Positive (TP): 88

```

[10]: # =====
# 9) Predict on New Sample
# =====

sample = x_test[0].reshape(1, -1)
prediction = nb_model.predict(sample)

print("\nPrediction for a new sample:",
      data.target_names[prediction][0])

```

Prediction for a new sample: benign

```
[ ]:
```